

Exam Points

```
interface A { default void act() { System.out.println("A"); } }
interface B extends A { default void act() { System.out.println("B"); } }
abstract class C implements A { }
class D extends C implements B { }
class Test {
    public static void main(String[] args) {
        new D().act();
    }
}
```

Output: **B**

SCENARIO 1: If a class implements two interfaces, and **one provides a default implementation** of a method while the **other declares it abstract**, the class **must explicitly resolve the conflict** by overriding the method.

SCENARIO 2: Two Interfaces Provide Default Implementations of the Same Method

```
interface A { default void say() { System.out.println("A"); } }
interface B { default void say() { System.out.println("B"); } }
class C implements A, B { }
```

Java doesn't know which default method to use — you **must override** the method and resolve the ambiguity.

```
class C implements A, B { public void say() {
    A.super.say(); // or B.super.say() }
}
```

SCENARIO 3: One Interface Provides a Default, the Class Implements It Directly

```
interface A { default void say() { System.out.println("A"); } }
class C implements A { // no override}
public class Test {
    public static void main(String[] args) {
        new C().say(); //  Output: A
    }
}
```

 **Compiles and runs** — no conflict.

SCENARIO 4: A Class Extends a Superclass That Defines the Method, and Implements an Interface with a Default Method

 **Rule: Class methods always win over interface default methods.**

If a class in the inheritance chain defines the method, it overrides the interface default, even if it's not overridden in the subclass.

```
class Super { public void say() { System.out.println("Super"); } }
interface A { default void say() { System.out.println("A"); } }
class C extends Super implements A { }
public class Test {
    public static void main(String[] args) {
        new C().say(); //  Output: Super
    }
}
```

SCENARIO 5: Interface Inheritance — Subinterface Overrides Default

 **Rule: The most specific default method wins.**

```
interface A { default void say() { System.out.println("A"); } }
interface B extends A { default void say() { System.out.println("B"); } }
class C implements B { }
public class Test { public static void main(String[] args) {
    new C().say(); //  Output: B
}
```

```
}}
```

SCENARIO 6: Abstract Class Implements Conflicting Interfaces But Doesn't Resolve the Conflict

✅ Must resolve the conflict, either by:

- Making `say()` abstract in `C`, or
- Providing a concrete implementation in `C`.

```
interface A { default void say() { System.out.println("A"); }}
interface B { void say();}
abstract class C implements A, B { } // ❌ Compilation Error
```

SCENARIO 7: Diamond Problem with Interfaces

✅ Rule: If both interfaces inherit the same default method from a common ancestor, and don't override it, no conflict arises.

```
interface A { default void say() { System.out.println("A"); }}
interface B extends A { }
interface C extends A { }
class D implements B, C { }
public class Test { public static void main(String[] args) {
    new D().say(); // ✅ Output: A
}}
```

Summary of Rules

Conflict Type	Requires Override?	Fix
Abstract vs Default	✅ Yes	Must override in the class
Default vs Default	✅ Yes	Must override and choose one explicitly
Class method vs Interface default	❌ No	Class method takes precedence
Interface inheritance with same default	❌ No	No conflict
Abstract class implements conflicting interfaces	✅ Yes	Override or declare abstract

✅ Tips to Resolve Conflicts

1. **Always check method signatures:** Conflicts only occur when signatures match.
2. Use `InterfaceName.super.method()` to disambiguate.
3. **Class methods override interface defaults** automatically.
4. **Be explicit:** even if you think one default is enough, Java might not agree!